Universidade Federal de Sergipe - UFS
Departamento de Sistemas de Informação - Itabaiana - DSI/Ita

Estruturas de Dados I - SINF0006

Exercício 02 - TAD - Listas Lineares

Implementação por meio de APONTADORES

Prof. Dr. Alcides Xavier Benicasa

Exercício **INDIVIDUAL AVALIATIVO**
Formato de entrega: ***código-fonte***
Envio: encaminhar arquivos via ***SIGAA***
Assunto: TAD - Listas Lineares - Apontadores

QUESTÃO 01: Considerando a implementação de **LISTAS Lineares por meio de ARRANJOS** apresentada na resolução do exercício anterior, desenvolva uma função adicional que permita a **Inserção Ordenada** de elementos na lista.

A lista deve ser mantida em **ordem crescente** de acordo com o campo `Chave` da estrutura `TInfo`.

Implemente a seguinte função:

```
InsereOrdenado (Lista, Item);
```

Requisitos:

- A função deve inserir o novo elemento na posição correta, garantindo que a lista permaneça ordenada.
- Não é permitido simplesmente inserir no final e ordenar posteriormente.
- Utilize deslocamento de elementos, quando necessário, conforme feito nas funções anteriores.
- Considere os casos de lista vazia e lista cheia.

Exemplo:
Dada a lista:

10 20 30 50

Ao inserir o elemento com chave 25, a lista deve ficar:

10 20 25 30 50

QUESTÃO 02: Considerando os conceitos sobre **LISTAS** Lineares apresentado em sala, e o código fonte *parcial* anexo neste exercício, implemente-a por meio de **APONTADORES** na linguagem *C*, respeitando as seguintes assinaturas:

```
InserePosicao (Lista,p,Item);
InsereFinal (Lista,Item);
RemoveFinal (Lista);
Pesquisa_Remove (Lista,Item);
```

main.cpp

```
#include <iostream>
#include <locale.h>
#include <stdlib.h>
using namespace std;
#include "LISTA_APONTADOR.h"
enum Escolhas {CRIA, INSEREINICIO, INSEREPOSICAO, INSEREFINAL, REMOVEINICIO,
    REMOVEFINAL, PESQUISAREMOVE, IMPRIME, FIM};

//-----------------------------------------------------------------------
Escolhas enterChoice(TLista& lista)
{
        system("cls");
        Imprime(lista);
        int menuChoice;
        cout << "\nMenu:" << endl
        << "1 - Insere no Início\n"
        << "2 - Insere na Posição\n"
        << "3 - Insere no Final\n"
        << "4 - Remove do Início\n"
        << "5 - Remove do Final\n"
        << "6 - Pesquisa e Remove\n"
        << "7 - Imprime\n"
        << "8 - Fim do programa\n"
        << "Opção: ";
        cin >> menuChoice;
        return (Escolhas) menuChoice;
}

void setDados(TInfo& item){
        cout << "\nInforme o Código: ";
        cin >> item.Chave;
}

void setPosicao(int& pos){
        cout << "\nInforme a Posição: ";
        cin >> pos;
}

int main()
{
        setlocale (LC_ALL,"portuguese");
        TInfo item;
        int p;
        TLista l;
        Cria(l);
        Imprime(l);

        Escolhas opcao;
        while ((opcao = enterChoice(l)) != FIM)
        {
                switch (opcao)
                {
                        case INSEREINICIO:
                        setDados(item);
                        InsereInicio(l,item);
                        break;
                        case INSERE POSICAO:
                        //setDados(item);
                        //setPosicao(p);
                        //InserePosicao(l,p,item);
                        break;
```

```
                                case INSEREFINAL:
                                //setDados(item);
                                //InsereFinal(l,item);
                                break;
                                case REMOVEINICIO:
                                RemoveInicio(l);
                                break;
                                case REMOVEFINAL:
                                //RemoveFinal(l);
                                break;
                                case PESQUISAREMOVE:
                                //setDados(item);
                                //PesquisaRemove(l,item.Chave);
                                break;
                                case IMPRIME:
                                Imprime(l);
                                break;
                                default:
                                cerr << "Opção incorreta\n";
                                break;
                        }
                }
                return 0;
        }
```

LISTA_APONTADOR.h

```
        //----LISTA IMPLEMENTADA POR MEIO DE APONTADOR----
        typedef struct NodoLista *Apontador;

        typedef struct  {
                int Chave;
                /*outros componentes*/
        } TInfo;

        struct NodoLista{
                TInfo Item;
                struct NodoLista *Proximo;
        };

        typedef struct {
                struct NodoLista *Primeiro;
                struct NodoLista *Ultimo;
        } TLista;

        //---------------------------------------------
        void Cria (TLista& Lista) {
                Lista.Primeiro = (Apontador) malloc(sizeof(struct NodoLista));
                Lista.Ultimo = Lista.Primeiro;
                Lista.Primeiro->Proximo = NULL;
        }

        //---------------------------------------------
        bool Vazia (TLista Lista) {
                return Lista.Primeiro==Lista.Ultimo?true:false;
        }

        //---------------------------------------------
        void InsereInicio (TLista& Lista, TInfo Item) {
                Apontador p = (Apontador) malloc(sizeof(struct NodoLista));
                p->Item = Item;
                p->Proximo = Lista.Primeiro->Proximo;
                Lista.Primeiro->Proximo = p;

                if (p->Proximo == NULL)
                Lista.Ultimo = p;
        }

        //---------------------------------------------
        void RemoveInicio (TLista& Lista) {
                if (Vazia(Lista))
                cout << "Erro : A lista está vazia.\n" << endl;
                else {
                        Apontador p = Lista.Primeiro->Proximo;
```

```
                        cout << "Elemento Removido = " << p->Item.Chave << endl;
                        Lista.Primeiro->Proximo = p->Proximo;
                        if (Lista.Primeiro->Proximo == NULL)
                        Lista.Ultimo = Lista.Primeiro;
                        free(p);
                }
        }
        //---------------------------------------------
        void Imprime (TLista Lista) {
                cout << "Itens na LISTA: " << endl;
                if (Vazia(Lista))
                cout << "Lista está vazia." << endl;
                else {
                        Apontador aux;
                        aux = Lista.Primeiro->Proximo;
                        while (aux != NULL)
                        {
                                cout << aux->Item.Chave << " ";
                                aux = aux->Proximo;
                        }
                        cout << endl;
                }
        }
        //---------------------------------------------
```